

CS4234: Optimization Algorithms
Semester 1, AY 2026-27
School of Computing, NUS

Solutions for Tutorial 1

Question 1. We are given an undirected graph $G = (V, E)$, and we wish to remove as few edges as possible so that the remaining graph is triangle-free.

Algorithm. Construct a maximal collection $\mathcal{T}$ of pairwise edge-disjoint triangles. This can be done greedily: examine all triples of vertices, and whenever the triple induces a triangle none of whose edges belongs to a previously selected triangle, add it to $\mathcal{T}$. Return

$$F = \bigcup_{T \in \mathcal{T}} E(T),$$

that is, remove all three edges of every selected triangle.

Lemma 1. *The graph $G - F$ is triangle-free.*

Proof. Suppose, for contradiction, that $G - F$ contains a triangle T . None of the edges of T belongs to F , so T is edge-disjoint from every triangle in $\mathcal{T}$. We could therefore add T to $\mathcal{T}$, contradicting the maximality of $\mathcal{T}$. $\square$

Lemma 2. *If F^* is an optimal set of edges whose removal destroys all triangles, then $|F^*| \geq |\mathcal{T}|$.*

Proof. The set F^* must contain at least one edge from each triangle in $\mathcal{T}$. The triangles in $\mathcal{T}$ are edge-disjoint, so the edges chosen for distinct triangles must be distinct. Hence $|F^*| \geq |\mathcal{T}|$. $\square$

Since the algorithm removes exactly three edges per selected triangle,

$$|F| = 3|\mathcal{T}| \leq 3|F^*| = 3\text{OPT}.$$

Thus the algorithm is a 3-approximation. If the graph has no triangle, both the algorithm and the optimum remove zero edges.

Using an adjacency matrix, we can examine all triples in $O(|V|^3)$ time. For each edge, we store one bit recording whether it already belongs to a selected triangle, so testing and selecting a triple takes constant time. The algorithm therefore runs in polynomial time; its running time is $O(|V|^3 + |E|)$.

Question 2A. For item i , let w_i and v_i denote its positive integral weight and nonnegative integral value. Recall that $V = \max_i v_i$. Since the value of every item is at most V , the value of any subset is at most nV .

For $i \in \{0, \dots, n\}$ and $p \in \{0, \dots, nV\}$, define

$D[i, p]$ = the minimum total weight of a subset of items $1, \dots, i$ having total value exactly p .

If no such subset exists, set $D[i, p] = +\infty$. Initialize

$$D[0, 0] = 0, \quad D[0, p] = +\infty \quad \text{for } p > 0.$$

For $i = 1, \dots, n$, use the recurrence

$$D[i, p] = \begin{cases} \min\{D[i-1, p], w_i + D[i-1, p - v_i]\}, & p \geq v_i, \\ D[i-1, p], & p < v_i. \end{cases}$$

The answer is the largest p such that $D[n, p] \leq W$.

Lemma 3. For every i and p , the table entry $D[i, p]$ has the meaning stated above.

Proof. We use induction on i . The boundary conditions make the claim true for $i = 0$. Consider a minimum-weight subset of items $1, \dots, i$ having value exactly p . If it omits item i , its weight is at least $D[i-1, p]$. If it includes item i , the remaining items have value $p - v_i$, and its weight is at least $w_i + D[i-1, p - v_i]$. Conversely, each finite alternative in the recurrence describes a subset of value exactly p . Taking the smaller alternative is therefore correct. $\square$

A subset is feasible precisely when its weight is at most W . It follows from the lemma that the largest p with $D[n, p] \leq W$ is the maximum value of a feasible subset. To recover the subset, backtrack from this table entry: compare $D[i, p]$ with $D[i-1, p]$; when the second alternative was used, include item i and replace p by $p - v_i$.

There are $O(n^2V)$ table entries, each computable in constant time. Hence the running time is $O(n^2V)$, and the space usage is $O(n^2V)$. If only the optimum value is needed, two rows suffice, and the space can be reduced to $O(nV)$.

Question 2B. First discard every item with $w_i > W$, since such an item cannot occur in a feasible solution. If no items remain, return the empty set. Redefine n to be the number of remaining items and $V = \max_i v_i$. If $V = 0$, every feasible solution has value zero, so again return the empty set. Otherwise, fix $0 < \epsilon < 1$ and set

$$K = \frac{\epsilon V}{n}, \quad \widehat{v}_i = \left\lfloor \frac{v_i}{K} \right\rfloor.$$

Run the dynamic program from Question 2A using the integer values $\widehat{v}_i$ and the original weights, and return a feasible set S of maximum scaled value.

Because $v_i \leq V$,

$$\widehat{v}_i \leq \left\lfloor \frac{V}{K} \right\rfloor = \left\lfloor \frac{n}{\epsilon} \right\rfloor.$$

Consequently, the running time from Question 2A becomes

$$O\left(n^2 \max_i \widehat{v}_i\right) = O\left(\frac{n^3}{\epsilon}\right),$$

which is polynomial in both n and $1/\epsilon$. The scaling can be implemented using integer arithmetic, for example by computing $\hat{v}_i = \lfloor nv_i/(\epsilon V) \rfloor$ when ϵ is represented as a rational number.

It remains to prove the approximation guarantee. Let S^* be an optimal solution for the original instance. By optimality of S for the scaled instance,

$$\sum_{i \in S} \hat{v}_i \geq \sum_{i \in S^*} \hat{v}_i.$$

For every item,

$$K\hat{v}_i \leq v_i < K(\hat{v}_i + 1).$$

Therefore,

$$\begin{aligned} \sum_{i \in S} v_i &\geq K \sum_{i \in S} \hat{v}_i \\ &\geq K \sum_{i \in S^*} \hat{v}_i \\ &> \sum_{i \in S^*} v_i - K|S^*| \\ &\geq \text{OPT} - nK = \text{OPT} - \epsilon V. \end{aligned}$$

Every remaining item fits individually, so an item of value V is a feasible solution. Hence $V \leq \text{OPT}$, and thus

$$\sum_{i \in S} v_i \geq \text{OPT} - \epsilon V \geq (1 - \epsilon)\text{OPT}.$$

The algorithm is therefore an FPTAS for Knapsack.

Question 3. For every integer $q \geq 1$, let G_q consist of q pairwise disjoint edges together with one isolated vertex z . The case $q = 5$ is shown below.

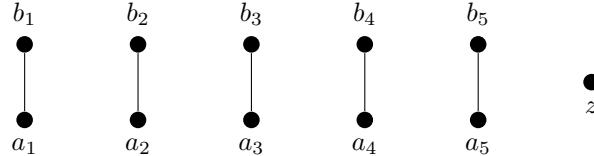


Figure 1: Five disjoint edges and one isolated vertex.

Every maximal matching in G_q contains all q edges: if one of these edges were omitted, neither of its endpoints could be matched by another edge, so it could be added to the matching. Consequently, **Greedy-Match-VC** places both endpoints of every edge in C . It does not place the isolated vertex in C . The independent set returned by **Greedy-IS** is therefore

$$I = V \setminus C = \{z\},$$

which has size one.

On the other hand, a maximum independent set contains z and exactly one endpoint of each of the q disjoint edges. Thus

$$|I^*| = q + 1, \quad |I| = 1, \quad \frac{|I^*|}{|I|} = q + 1.$$

Since q can be arbitrarily large, the approximation ratio is unbounded. In particular, complementing the vertex cover returned by a 2-approximation does not yield any constant-factor approximation for Maximum Independent Set.

The underlying reason is that complementation subtracts cardinalities rather than preserving their ratio. If C^* is a minimum vertex cover and $I^* = V \setminus C^*$, the guarantee $|C| \leq 2|C^*|$ gives only

$$|V \setminus C| \geq |V| - 2|C^*| = 2|I^*| - |V|,$$

and this lower bound may be zero even when $|I^*|$ is large.